

KLE'S GH BCA COLLEGE, HAVERI

Department of Computer Applications

FundCircle

Pigmy Savings and Loan Collection Management System

A Project Report

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Applications (BCA)

Under the Guidance of

Prof. Abhishek N. G.

Academic Year 2024 - 2025

CHAPTER 1

INTRODUCTION

1.1 Introduction

FundCircle is a secure, cloud-native multi-tenant web application purpose-built for Pigmy savings operators, microfinance firms, chit fund businesses, and NBFC companies operating across India. It digitizes the complete lifecycle of Pigmy savings collection and loan management — replacing paper passbooks, manual ledgers, and cash-based workflows with a role-stratified web application accessible from any modern browser.

The platform is architected around three distinct user roles within each organization: Organization Owner, Agent, and Customer. Each role receives a dedicated browser panel with permissions scoped precisely to their operational responsibilities. Organization Owners provision and manage Agents and Customers; Agents conduct daily savings collections and EMI collections; Customers access their passbook, savings balance, loan schedule, and receipt history.

FundCircle is built on a modern full-stack: React with Vite and TypeScript for the frontend, Node.js and Express.js for the backend API, Firebase Firestore as the NoSQL cloud database, and Clerk Authentication for identity and session management. The platform delivers daily, weekly, and monthly savings plans, full loan lifecycle management from application to closure, digital receipts, passbooks, and comprehensive reporting — all within a strictly tenant-isolated data architecture.

1.2 Existing System and Problem Statement

The existing Pigmy collection ecosystem in India relies overwhelmingly on paper-based processes. Agents carry physical passbooks to customer locations, manually record deposits in handwritten ledgers, and transport cash to the organization office at day's end. Loan disbursements and EMI collections are similarly tracked on paper or in simple spreadsheets, with no centralized visibility.

This manual model introduces serious operational vulnerabilities: handwritten entries are susceptible to alteration, loss, and arithmetic error. Organization owners have no real-time dashboard of collections, loan outstanding, or agent performance. Customers cannot independently verify their savings balance or EMI schedule. Agent-level fraud — underreporting collections, falsifying receipts, or misappropriating cash — is difficult to detect without an immutable audit trail.

There is no digitized loan management workflow: loan applications, approvals, disbursements, EMI scheduling, and closures are all handled informally, making loan portfolio visibility impossible.

1.3 Proposed System

FundCircle replaces every manual touchpoint in the Pigmy savings and loan workflow with a digital, automated, and auditable equivalent. Organization Owners access a multi-tenant dashboard with real-time KPIs: total savings, total loans outstanding, today's collections, active agents, active customers, pending EMIs, overdue loans, and a monthly collection summary chart.

Agents access a dedicated Agent Dashboard showing their assigned customers, today's savings collections, EMI collections, pending visits, and recent transaction history. The savings collection screen enables rapid recording of daily, weekly, or monthly deposits with instant receipt generation.

Customers access their own portal showing savings balance, loan balance, EMI schedule, collection history, digital receipts, and a full passbook. All user accounts are managed exclusively through Clerk Authentication; neither Agents nor Customers can self-register. Organization Owners create all accounts through the management panels, and the system automatically provisions Clerk credentials and generates a secure temporary password.

1.4 Objectives

The primary objectives of FundCircle are:

- **Digitize Pigmy Savings Collection:** Replace paper passbooks and manual ledgers with real-time digital collection recording for daily, weekly, and monthly savings plans.
- **Loan Lifecycle Management:** Provide a complete loan workflow covering application, approval, disbursement, EMI scheduling, EMI collection, outstanding tracking, and loan closure.
- **Multi-Tenant Architecture:** Deliver a single platform serving multiple organizations simultaneously with complete tenant-level data isolation.
- **Role-Based Access Control:** Enforce strict permission boundaries across Organization Owner, Agent, and Customer roles managed through Clerk Authentication and Firestore organization membership.
- **Digital Receipts and Passbooks:** Generate itemized receipts for every collection and maintain a complete digital passbook for each customer account.
- **Real-Time Dashboards:** Provide role-appropriate dashboards with live KPIs for owners, agents, and customers.
- **Fraud Elimination:** Maintain immutable audit logs for every transaction, making collection discrepancies detectable at the agent, customer, and organization level.
- **Reporting and Analytics:** Generate monthly collection summaries, loan portfolio reports, and EMI aging reports.

1.5 Advantages

Key advantages of the FundCircle platform include:

- **Zero Paper Dependency:** All operations — savings collection, loan processing, EMI collection, receipt generation — are entirely digital, eliminating paper costs and manual errors.
- **Secure Authentication:** Clerk Authentication with secure session management and JWT-based tokens provides robust identity management for all user roles.
- **Scalable SaaS Architecture:** Multi-tenant design supports multiple organizations with complete data isolation and a modern cloud-native approach.
- **Controlled User Onboarding:** Only Organization Owners can create Agent and Customer accounts. The system auto-provisions Clerk credentials, ensuring controlled and auditable onboarding.
- **Complete Loan Visibility:** End-to-end loan management with auto-generated EMI schedules, real-time outstanding balance tracking, and automated loan closure upon full repayment.
- **Customer Transparency:** Customers access their complete savings passbook, loan details, EMI schedule, and receipt history through a secure portal at any time.

- Tenant Isolation: Every query is organization-scoped, ensuring zero cross-tenant data exposure.
- Realtime Updates: Firebase Firestore enables live dashboard updates without manual refresh.

CHAPTER 2

HARDWARE AND SOFTWARE REQUIREMENTS

2.1 Hardware Requirements

2.1.1 Development Machine Requirements

Component	Minimum Specification	Recommended Specification
Processor	4-Core CPU @ 2.0 GHz	8-Core CPU @ 3.0 GHz or higher
RAM	8 GB DDR4	16 GB DDR4 or higher
Storage	100 GB SSD	256 GB SSD or higher
Operating System	Windows 10 / macOS 12 / Ubuntu 20.04	Windows 11 / macOS 14 / Ubuntu 22.04
Internet Connection	10 Mbps broadband	50 Mbps broadband or higher

2.1.2 Client-Side Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor	Intel Core i3 / AMD Ryzen 3	Intel Core i5 / AMD Ryzen 5 or higher
RAM	4 GB DDR4	8 GB DDR4 or higher
Storage	128 GB SSD / HDD	256 GB SSD or higher
Display	1366 x 768 resolution	1920 x 1080 Full HD or higher
Internet Connection	5 Mbps broadband	25 Mbps broadband or higher
Browser	Chrome 110+ / Firefox 108+	Latest Chrome / Edge / Firefox

2.1.3 Field Agent Device Requirements (Mobile / Tablet)

Component	Minimum Specification	Recommended Specification
Device Type	Android 9.0+ Smartphone	Android 12+ Smartphone / Tablet
Processor	Octa-Core 1.6 GHz	Octa-Core 2.0 GHz or higher
RAM	3 GB	4 GB or higher
Storage	32 GB Internal	64 GB or higher
Internet	4G LTE	4G LTE / 5G
Browser	Chrome for Android	Chrome for Android (Latest)

2.2 Software Requirements

2.2.1 Frontend Technologies

Technology	Version	Purpose	License
Vite	5.x	Ultra-fast frontend build tool with Hot Module Replacement	MIT
React	18.x	Component-based declarative UI framework	MIT
TypeScript	5.x	Strongly-typed JavaScript superset	Apache 2.0
Tailwind CSS	3.x	Utility-first CSS framework for rapid UI development	MIT
React Router DOM	6.x	Client-side routing and navigation	MIT

2.2.2 Backend Technologies

Technology	Version	Purpose	License
Node.js	20.x LTS	Server-side JavaScript runtime environment	MIT
Express.js	4.x	RESTful API framework for Node.js	MIT
TypeScript	5.x	Type-safe server-side development	Apache 2.0
Firebase Admin SDK	12.x	Server-side access to Firebase Firestore	Apache 2.0

2.2.3 Database

Technology	Version	Purpose	License
Firebase Firestore	Cloud (NoSQL)	Primary cloud NoSQL document database for all application data	Proprietary (Google)
Firebase Admin SDK	12.x	Server-side Firestore read/write operations	Apache 2.0

2.2.4 Authentication and Security

Technology / Tool	Purpose
Clerk Authentication	Complete identity and session management for all three user roles
Clerk User Provisioning API	Programmatic creation of Agent and Customer Clerk accounts by the Owner
Clerk Organization	Tenant-scoped membership management for multi-

Membership	organization isolation
JWT Session Tokens	Secure, stateless session management for all authenticated users
Express.js Middleware	Authentication, Tenant Resolution, and RBAC enforcement on every protected route

2.2.5 Development Tools

Tool	Purpose
VS Code	Primary integrated development environment
GitHub	Version control and collaborative development
ESLint / Prettier	Static code analysis and automated code formatting
Postman	REST API endpoint testing and documentation
Node.js npm	Package management for both frontend and backend dependencies
Firebase Console	Firestore data browsing, security rules management, and monitoring
Clerk Dashboard	User management, organization management, and session monitoring

CHAPTER 3

SYSTEM DESIGN

3.1 Design Overview

FundCircle's system design is founded upon a modular backend combined with a component-driven Single Page Application (SPA) on the frontend, communicating through a RESTful API layer. The design enforces strict separation of concerns, type safety throughout the full stack, and a multi-tenant data model that guarantees organizational isolation at every system layer.

The multi-tenant architecture is implemented through Clerk's organizational session management and a tenant-context middleware chain in the Express.js backend. Every protected API request passes through three middleware layers — Authentication, Tenant Resolution, and RBAC — before reaching a route handler. All Firestore queries include a mandatory organizationId scope, ensuring no query can read, write, or modify data outside the authenticated tenant boundary.

User management follows a controlled provisioning model: Organization Owners are the sole authority for creating Agent and Customer accounts. Neither role can self-register. The Owner enters an email address in the respective management panel; the backend calls the Clerk Admin API to create the Clerk user, generates a cryptographically random temporary password, creates the Firestore records, and dispatches a welcome email containing the login URL and temporary password.

3.2 System Architecture

The FundCircle system architecture comprises six distinct layers, each with well-defined responsibilities and strict interface boundaries:

Layer Component	Layer Name
React + Vite + TypeScript SPA Tailwind CSS React Router DOM	CLIENT PRESENTATION LAYER
Clerk Auth (Owner / Agent / Customer) Role-Based Route Guards Session Management	AUTHENTICATION & ROUTING LAYER
Express.js REST API (Node.js 20 LTS) Auth → Tenant → RBAC → Route Handler Middleware Chain	API GATEWAY LAYER
OrganizationService AgentService CustomerService SavingsService LoanService CollectionService AuditService	BUSINESS LOGIC / SERVICE LAYER
Firestore Service Layer — Organization-Scoped Document Access Collection Queries	DATA ACCESS LAYER
Firebase Firestore (Cloud NoSQL) Realtime Updates Organization-Scoped Collections	DATA PERSISTENCE LAYER

3.2.1 Architecture Flow Diagram

The following diagram illustrates the end-to-end request and data flow through the FundCircle architecture:

Ste	Component	Description
-----	-----------	-------------

p		
1	Owner / Agent / Customer (Browser)	User interacts with the React SPA on any modern browser
2	React Frontend (Vite + TypeScript)	SPA renders role-appropriate UI, manages client state
3	Clerk Authentication	Validates session, attaches JWT with organization context
4	Express.js API Layer	Receives request, runs Auth → Tenant → RBAC middleware chain
5	Business Logic Layer	Service classes execute domain operations
6	Firebase Firestore	Persists or retrieves organization-scoped documents
7	Realtime Dashboard Updates	Firestore listeners push live updates back to the React frontend

3.3 Module Description

3.3.1 Authentication Module

Manages the complete identity lifecycle for all platform users through Clerk Authentication. Organization Owners register via email and password directly on the Clerk-hosted sign-in interface. Agents and Customers are provisioned programmatically by the Owner: the system calls the Clerk Admin API, creates the Clerk user with a system-generated temporary password, and dispatches a welcome email. On first login, users are prompted to change their temporary password.

3.3.2 Organization Module

Manages the complete lifecycle of tenant organizations on the platform. Each organization maintains its own Clerk Organization and a corresponding Firestore organizations document. The module enforces strict tenant isolation: every API request is resolved to its organization context through the Clerk session, and all downstream Firestore queries are scoped to that organization's data.

3.3.3 Agent Management Module

Organization Owners manage the complete agent lifecycle through the Agent Management panel. Owners enter an agent's email address; the system creates the Clerk user, generates a secure random password, creates the Agent document in Firestore under the organization's scope, and sends a welcome email. The agent logs in with their temporary password and completes their profile.

3.3.4 Customer Management Module

Organization Owners create and manage customer accounts through the Customer Management panel. Owners enter the customer's email address; the system provisions a Clerk user, generates credentials, creates the Customer document in Firestore with a default agent assignment, and dispatches a welcome email. Customers complete their profile, including savings plan selection, on first login.

3.3.5 Pigmy Savings Module

The Pigmy Savings module is the primary operational workflow for agents. It supports daily, weekly, and monthly savings plans. Agents record collection entries against customer savings accounts in real time.

Each collection entry updates the savings account balance, generates a unique receipt number, and writes an immutable audit log entry to Firestore. Customers can view their full transaction history, passbook, and digital receipts through the Customer Dashboard.

3.3.6 Loan Management Module

The Loan Management module covers the full loan lifecycle. Loan applications are submitted through the Owner Dashboard, reviewed, and approved or rejected. Upon approval, the system records the disbursement amount, generates an EMI schedule based on principal, interest rate, and tenure, and activates the loan in Firestore. Agents record EMI collections against active loans; each payment updates the outstanding balance and marks the installment as paid.

3.3.7 Dashboard Module

Delivers role-appropriate real-time operational intelligence for each user type. The Owner Dashboard aggregates live KPIs: total savings across all customers, total loan portfolio, today's collections, active customers, active agents, pending EMIs, overdue loans, and a monthly collection summary chart. The Agent Dashboard shows assigned customers, today's savings collections, EMI collections, and recent transactions. The Customer Dashboard shows savings balance, loan balance, EMI schedule, collection history, receipts, and passbook. Firestore's real-time listeners enable dashboard data to update without manual page refresh.

3.3.8 Reporting Module

Provides organization-level reports for operational decision-making. Reports include monthly collection summaries with agent-level breakdowns, loan portfolio reports showing disbursed vs. outstanding amounts, EMI aging reports categorizing installments by days overdue, customer savings growth trajectories, and daily collection reconciliation summaries per agent.

3.3.9 Audit Log Module

Every state-changing operation — collection recording, loan approval, EMI collection, agent creation, customer creation, account status changes — writes an immutable audit log document to the `audit_logs` Firestore collection, capturing the actor identity, action type, affected entity, old value, new value, and timestamp. Audit logs are tenant-scoped and accessible to Organization Owners.

3.4 Database Design — Firestore Collections

FundCircle uses Firebase Firestore as its primary database. Data is organized into the following top-level collections, with every document including an `organizationId` field for multi-tenant isolation.

organizations

Field	Type	Description
id	String (Auto-ID)	Unique Firestore document ID
name	String	Organization display name
clerkOrgId	String	Clerk organization reference (Unique)
status	String	ACTIVE or SUSPENDED
createdAt	Timestamp	Record creation timestamp

updatedAt	Timestamp	Last modification timestamp
-----------	-----------	-----------------------------

organization_members

Field	Type	Description
id	String (Auto-ID)	Unique document ID
user_id	String	Clerk user ID of the member
organization_id	String	Reference to the organizations collection
role	String	OWNER, AGENT, or CUSTOMER (scoped to this organization)

Note: Roles are always organization-scoped. A user can be an AGENT in one organization and a CUSTOMER in another. Authorization always reads membership.role for the active organization, never a global role.

users

Field	Type	Description
id	String (Auto-ID)	Firestore document ID
clerkUserId	String	Clerk user reference (Unique)
organizationId	String	Tenant association
email	String	User email address
firstName	String	User first name
lastName	String	User last name
createdAt	Timestamp	Record creation timestamp

agents

Field	Type	Description
id	String (Auto-ID)	Unique agent document ID
organizationId	String	Tenant association
clerkUserId	String	Clerk user reference
email	String	Agent email address
firstName	String	Agent first name
lastName	String	Agent last name
phone	String	Agent contact number (optional)
status	String	ACTIVE or DEACTIVATED
createdAt	Timestamp	Record creation timestamp

customers

Field	Type	Description
id	String (Auto-ID)	Unique customer document ID
organizationId	String	Tenant association
agentId	String	Assigned agent reference
clerkUserId	String	Clerk user reference
email	String	Customer email address
accountNumber	String	Auto-generated account number (Unique)
firstName	String	Customer first name
lastName	String	Customer last name
phone	String	Customer contact number (optional)
address	String	Customer residential address (optional)
status	String	ACTIVE, DORMANT, or CLOSED

savings_accounts

Field	Type	Description
id	String (Auto-ID)	Unique savings account document ID
customerId	String	Reference to customers collection (1:1)
organizationId	String	Tenant scoping
planType	String	DAILY, WEEKLY, or MONTHLY
scheduledAmount	Number	Scheduled deposit per period
totalBalance	Number	Running cumulative savings balance (default: 0)
startDate	Timestamp	Plan commencement date
status	String	ACTIVE, SUSPENDED, or CLOSED

savings_transactions

Field	Type	Description
id	String (Auto-ID)	Unique transaction document ID
savingsAccountId	String	Reference to savings_accounts
organizationId	String	Tenant scoping
agentId	String	Recording agent reference
amount	Number	Deposit amount (must be > 0)
balanceAfter	Number	Savings balance after transaction
receiptNo	String	Auto-generated receipt number (Unique)
collectedAt	Timestamp	Collection timestamp

loans

Field	Type	Description
id	String (Auto-ID)	Unique loan document ID
organizationId	String	Tenant scoping
customerId	String	Borrower reference
principalAmount	Number	Loan disbursement amount
interestRate	Number	Annual interest rate (%)
tenureMonths	Number	Loan tenure in months
emiAmount	Number	Monthly EMI amount (auto-calculated)
disbursedAt	Timestamp	Disbursement timestamp (nullable)
status	String	PENDING, ACTIVE, CLOSED, or REJECTED
outstandingBalance	Number	Current outstanding balance

loan_installments

Field	Type	Description
id	String (Auto-ID)	Unique installment document ID
loanId	String	Parent loan reference
organizationId	String	Tenant scoping
installmentNo	Number	Installment sequence number
dueDate	Timestamp	EMI due date
emiAmount	Number	Scheduled EMI amount
paidAmount	Number	Amount paid (default: 0)
paidAt	Timestamp	Payment timestamp (nullable)
status	String	PENDING, PAID, or OVERDUE

collections

Field	Type	Description
id	String (Auto-ID)	Unique collection document ID
organizationId	String	Tenant scoping
agentId	String	Recording agent reference
customerId	String	Account holder reference
collectionType	String	SAVINGS or LOAN_EMI
referenceId	String	savings_transactions or loan_installments ID

amount	Number	Collected amount (must be > 0)
receiptNo	String	Auto-generated receipt number (Unique)
collectedAt	Timestamp	Collection timestamp

audit_logs

Field	Type	Description
id	String (Auto-ID)	Unique audit log document ID
organizationId	String	Tenant scoping
actorId	String	Clerk user ID of the acting user
actorRole	String	Role of the acting user
action	String	Action descriptor (e.g., AGENT_CREATED)
entityType	String	Entity type (Agent, Customer, Loan, etc.)
entityId	String	ID of the affected entity
metadata	Map	Additional context (old/new values, etc.) — optional
createdAt	Timestamp	Audit entry timestamp

3.5 ER Diagram

The Entity-Relationship diagram below describes the core data model and inter-entity relationships of FundCircle in Firebase Firestore:

Collection	Key Fields
organizations	id (PK), name, clerkOrgId, status, createdAt
organization_members	id (PK), user_id (FK → users), organization_id (FK → organizations), role
users	id (PK), organizationId (FK), clerkUserId, email (Unique), firstName, lastName
agents	id (PK), organizationId (FK), clerkUserId, email (Unique), firstName, lastName, phone, status
customers	id (PK), organizationId (FK), agentId (FK → agents), clerkUserId, email (Unique), accountNumber (Unique), firstName, lastName, status
savings_accounts	id (PK), customerId (Unique FK → customers), organizationId (FK), planType, scheduledAmount, totalBalance, startDate, status
savings_transactions	id (PK), savingsAccountId (FK), organizationId (FK), agentId (FK), amount, balanceAfter, receiptNo (Unique), collectedAt
loans	id (PK), organizationId (FK), customerId (FK), principalAmount, interestRate, tenureMonths, emiAmount, outstandingBalance, status

loan_installments	id (PK), loanId (FK), organizationId (FK), installmentNo, dueDate, emiAmount, paidAmount, paidAt, status
collections	id (PK), organizationId (FK), agentId (FK), customerId (FK), collectionType, referenceId, amount, receiptNo (Unique), collectedAt
audit_logs	id (PK), organizationId (FK), actorId, actorRole, action, entityType, entityId, metadata, createdAt

Key Relationships: One Organization has many members, agents, and customers. One Agent is assigned to many Customers and records many Collections. One Customer has one SavingsAccount and many Loans, many Collections, and many SavingsTransactions. One Loan has many LoanInstallments. All documents are tenant-scoped via organizationId, enforcing strict multi-tenant isolation at the data layer.

3.6 Use Case Diagram

Actor	Use Cases
Organization Owner	Create Agent, Deactivate Agent, Create Customer, View/Edit Customer, Record Savings Collection, Record EMI Collection, Approve/Reject Loan, View Owner Dashboard, Generate Reports, View Audit Logs
Agent	View Agent Dashboard, View Assigned Customers, Record Savings Collection, Record EMI Collection, Generate Receipt, View Collection History
Customer	View Customer Dashboard, View Savings Balance, View Passbook, View Loan Balance, View EMI Schedule, Download Receipt

3.7 Clerk ↔ Firestore Synchronization

Because FundCircle uses both Clerk (for authentication and session management) and Firebase Firestore (for application data persistence), it is essential that both systems remain in sync. The following flow describes how user creation propagates through both systems:

Step	System	Action
1	Clerk	Create Clerk User via Admin API (email + generated password)
2	Clerk	Create Organization Membership (links user to org with role)
3	Firestore	Create User document in users collection (organizationId scoped)
4	Firestore	Create Agent or Customer document with Clerk user ID reference
5	Firestore	Create SavingsAccount document (for Customer creation)
6	Dashboard	React frontend receives updated data via Firestore listeners
7	Isolation	All subsequent queries are scoped by organizationId

This synchronized flow ensures that every user visible in the React application has both a valid Clerk identity (for authentication) and a corresponding Firestore document (for application data). Deleting or

deactivating a user in either system is handled by the backend to maintain consistency across both platforms.

3.8 Organization Role Model

FundCircle implements an organization-scoped role model, meaning a user's role is always relative to a specific organization. A single user can hold different roles in different organizations simultaneously. For example, a person could be an AGENT in Organization A and a CUSTOMER in Organization B.

Roles are stored exclusively in the `organization_members` Firestore collection using the fields: `user_id`, `organization_id`, and `role`. A global role is never stored. Authorization on every protected API endpoint reads `membership.role` for the currently active organization from the Clerk session context.

Role	Description	Permissions
OWNER	Organization administrator	Full access: create agents and customers, approve loans, view reports, access audit logs
AGENT	Field collection agent	Record savings and EMI collections for assigned customers, generate receipts
CUSTOMER	Account holder	View own savings balance, passbook, loan details, EMI schedule, and receipts

CHAPTER 4 IMPLEMENTATION

4.1 Authentication Flow

4.1.1 Owner Registration Flow

Step	Description
1	Owner navigates to the FundCircle registration page
2	Owner registers via Clerk-hosted sign-up (email + password)
3	Clerk creates the user account and issues a session token
4	Backend creates the Organization document in Firestore
5	organization_members document created with role: OWNER
6	Owner is redirected to the Owner Dashboard

4.1.2 Agent Creation Flow (Owner-Initiated)

The agent creation algorithm executes the following sequence when an Organization Owner creates a new agent:

Step	Description
1	Owner submits the agent's email address via the Agent Management panel
2	Backend validates that the email is not already registered in Clerk
3	A cryptographically secure random password is generated using Node.js crypto.randomBytes
4	The Clerk Admin API is called to create a new Clerk user with the provided email and password
5	An agents document is created in Firestore linked to the organizationId and Clerk user ID
6	An organization_members document is created with role: AGENT
7	A welcome email is dispatched with the temporary password and login URL
8	An audit_logs document is written recording the creation event
9	Agent logs in, is prompted to complete profile and change password

4.1.3 Customer Creation Flow (Owner-Initiated)

Step	Description
1	Owner submits customer email and selects assigned agent

2	Backend validates email uniqueness in Clerk
3	Clerk user is created with a system-generated temporary password
4	A customers document is created in Firestore with organizationId and agentId
5	A savings_accounts document is created in ACTIVE status with default DAILY plan
6	An organization_members document is created with role: CUSTOMER
7	A welcome email is dispatched with login credentials
8	An audit_logs document is written
9	Customer logs in, completes profile, configures savings plan preference

4.2 Algorithms and Pseudocode

4.2.1 Savings Collection Recording

The savings collection algorithm executes the following sequence:

- (1) Agent submits customer ID and collection amount via the Savings Collection screen.
- (2) Tenant Middleware validates the agent is recording within their organization.
- (3) Agent scope validation confirms the customer is assigned to the requesting agent.
- (4) Amount is validated as a positive number.
- (5) A unique receipt number is generated using the pattern FC-{ORGSUG}-{YYYYMMDD}-{SEQUENCE}.
- (6) A savings_transactions document is created in Firestore with balanceAfter computed as totalBalance + amount.
- (7) The savings_accounts document's totalBalance field is updated.
- (8) A collections document is created with collectionType: SAVINGS.
- (9) An audit_logs document is written.
- (10) The digital receipt is returned to the agent interface for display.

4.2.2 Loan Approval and EMI Schedule Generation

The loan approval algorithm:

- (1) Owner reviews a PENDING loan application and approves it.
- (2) The system calculates the monthly EMI using the standard formula: $EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)$, where P is principal, r is monthly interest rate, and n is tenure in months.
- (3) loan_installments documents are created in Firestore for each EMI period with due dates computed from the disbursement date.
- (4) The loans document status is updated to ACTIVE and outstandingBalance is set to principal plus total interest.
- (5) An audit_logs document records the approval.
- (6) The Customer Dashboard immediately reflects the new loan and EMI schedule via Firestore listeners.

4.2.3 EMI Collection Recording

- (1) Agent selects an active loan customer and selects the overdue or next pending installment.
- (2) Tenant and agent scope validations are performed.
- (3) The loan_installments document is updated to PAID status with paidAt timestamp and paidAmount.
- (4) The loans document's outstandingBalance is decremented by the EMI amount.
- (5) A collections document is created with collectionType: LOAN_EMI.
- (6) If outstandingBalance reaches zero, the Loan status is updated to CLOSED automatically.
- (7) A receipt is generated and returned.
- (8) An audit_logs document is written.

4.2.4 Authentication Middleware Chain

Every protected API request passes through three Express.js middleware layers in sequence:

Middleware	Responsibility	Failure Response
Auth Middleware	Extracts Bearer token, verifies it with Clerk SDK, attaches verified user identity to request context	401 Unauthorized
Tenant Middleware	Extracts Clerk Organization ID from session, loads Firestore organization document, validates organization is ACTIVE, injects organizationId into request context	403 Forbidden
RBAC Middleware	Checks the user role from organization_members against the allowed roles for the endpoint	403 Forbidden

4.2.5 Dashboard Analytics Computation

Owner Dashboard analytics queries are composed in parallel for performance:

- (1) Total Savings: SUM of savings_accounts.totalBalance for the organization.
- (2) Total Loans: SUM of loans.principalAmount for ACTIVE loans.
- (3) Today's Collections: SUM and COUNT of collections with collectedAt >= today's midnight.
- (4) Active Customers: COUNT of customers with status ACTIVE.
- (5) Active Agents: COUNT of agents with status ACTIVE.
- (6) Pending EMIs: COUNT of loan_installments with status PENDING and dueDate <= today.
- (7) Overdue Loans: COUNT of distinct loans with at least one OVERDUE installment.
- (8) Monthly Collection Summary: GROUP BY calendar month for the last 12 months.

All queries are scoped by organizationId via Firestore where clauses.

4.2.6 Key Implementation Details — Tenant Isolation

Tenant isolation is implemented at two layers. At the session layer, Clerk includes the active Organization ID in the JWT token. At the data layer, a custom Express.js middleware resolves the Organization document from Firestore using this ID and injects the organizationId into every request object. All Firestore service methods include a mandatory `.where('organizationId', '==', organizationId)` clause. This dual-layer enforcement means that even if a request arrives with a valid token but references data from a different organization, the Firestore query will return no results, making tenant isolation transparent and absolute.

4.3 Savings Module

4.3.1 Savings Account Creation

When an Organization Owner creates a Customer, the backend automatically creates a `savings_accounts` document in Firestore linked to the new customer. The account is initialized with a DAILY plan type, a scheduled amount of 0 (to be configured by the customer on first login), and a `totalBalance` of 0.

4.3.2 Daily Collection

Agents record savings deposits through the Savings Collection screen. The agent searches for the customer, enters the deposit amount, and submits. The system creates a `savings_transactions` document, updates the `savings_accounts` balance, and generates a digital receipt with a unique receipt number.

4.3.3 Passbook

The Customer Dashboard displays a complete passbook view — a chronological ledger of all `savings_transactions` documents for the customer's savings account, showing date, amount deposited, and running balance after each transaction.

4.3.4 Transaction History and Balance Tracking

The `savings_accounts` collection maintains a `totalBalance` field that is updated on every collection event. The `savings_transactions` collection provides the full historical record. Both are queried together to render the passbook and balance displays.

4.3.5 Savings Reports

Organization Owners can generate savings reports showing monthly deposit totals per agent, customer savings growth over time, and daily reconciliation summaries. These reports are rendered using client-side aggregation of Firestore data with date-range filtering.

4.4 Loan Module

4.4.1 Loan Application

Organization Owners submit loan applications on behalf of customers through the Loan Management screen. The application captures principal amount, interest rate, and tenure in months. A `loans` document is created in Firestore with status `PENDING`.

4.4.2 Loan Approval

The Owner reviews pending loan applications and approves or rejects them. On approval, the system calculates the EMI amount, creates all loan_installments documents, and updates the loan status to ACTIVE.

4.4.3 Loan Disbursement

Upon approval, the disbursedAt timestamp is recorded and the outstandingBalance is initialized. The customer can immediately view the loan details and EMI schedule through their dashboard.

4.4.4 EMI Schedule

The EMI schedule is generated automatically upon loan approval. Each loan_installments document captures the installment number, due date, scheduled EMI amount, and initial PENDING status. The schedule is visible to both the Owner and the Customer.

4.4.5 Collection Tracking

Agents record EMI payments through the EMI Collection screen. Each payment updates the corresponding loan_installments document to PAID, decrements the loans.outstandingBalance, and creates a collections document.

4.4.6 Overdue Monitoring

Installments whose dueDate has passed and status remains PENDING are flagged as OVERDUE. The Owner Dashboard displays overdue loan counts, and the Loan Management screen highlights overdue installments in red.

4.4.7 Loan Reports

Loan portfolio reports show total disbursed amount, total outstanding, total repaid, and a breakdown by loan status (Active, Closed, Rejected). The EMI Aging Report categorizes overdue installments into buckets: 1-30 days, 31-60 days, 61-90 days, and 90+ days overdue.

CHAPTER 5

MULTI-TENANT SECURITY

5.1 Organization Isolation

Every Firestore document in every collection includes an `organizationId` field. All API read and write operations include a `.where('organizationId', '==', req.organizationId)` filter. This ensures that no API endpoint, regardless of how it is called, can return or modify data belonging to a different organization.

5.2 Role-Based Access Control

Role-based access is enforced by the RBAC middleware on every protected Express.js route. The user's role is read from the `organization_members` Firestore collection for the active organization. Attempting to access an owner-only endpoint with an Agent or Customer token returns 403 Forbidden.

5.3 Firestore Security Rules

Firestore Security Rules provide a second layer of protection directly at the database level. All read and write operations are restricted to authenticated users. Document-level rules verify that the requesting user's `organizationId` matches the document's `organizationId` field, preventing direct client-side access to cross-tenant data even if API middleware were bypassed.

5.4 Clerk Session Validation

The Auth Middleware uses the Clerk Node.js SDK to verify every incoming Bearer token. Expired tokens, tampered tokens, and tokens for deactivated users are all rejected at this layer with a 401 Unauthorized response before any business logic or Firestore query is executed.

5.5 Audit Logging

Every state-changing operation writes an immutable document to the `audit_logs` Firestore collection. This log captures the actor's Clerk user ID, their role, the action performed, the entity affected, and a metadata map with old and new values. Audit logs cannot be modified or deleted through the application; they serve as the primary mechanism for fraud detection and operational review.

5.6 Data Protection

Clerk handles all credential storage and session management, eliminating the need for the application to store passwords or manage session tokens directly. Firebase Firestore encrypts all data at rest and in transit by default. The application never stores plaintext credentials; temporary passwords are generated, dispatched by email, and immediately discarded from server memory.

CHAPTER 6

SYSTEM TESTING

6.1 Testing Overview

FundCircle implements a comprehensive multi-level testing strategy covering authentication lifecycle testing, tenant isolation validation, API integration testing, and end-to-end workflow testing simulating real-world user journeys. API endpoint testing uses Postman to validate authentication middleware, tenant injection, RBAC enforcement, and Firestore state mutations. Tenant isolation testing uses two-organization environments to validate that cross-tenant API requests return the expected 403 Forbidden response on every resource endpoint.

6.2 Test Cases

6.2.1 Authentication Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-001	Owner Login Success	Valid email and password	Session token returned; 200 OK	Pass
TC-002	Wrong Password	Correct email, wrong password	401 Unauthorized	Pass
TC-003	Suspended Organization Login	Valid credentials, suspended org	403 Forbidden	Pass
TC-004	Agent Login Success	Temporary password from welcome email	Session active; 200 OK	Pass
TC-005	Customer Login Success	Temporary password from welcome email	Session active; 200 OK	Pass
TC-006	Deactivated Agent Login	Valid credentials, deactivated account	403 Forbidden	Pass
TC-007	Expired Token Request	Expired JWT Bearer token	401 Unauthorized	Pass
TC-008	No Token Request	Request with no Authorization header	401 Unauthorized	Pass

6.2.2 Agent Creation Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-010	Valid Agent Creation	Owner token, valid new email	Clerk user created, Agent doc in Firestore, welcome	Pass

			email sent; 201	
TC-011	Duplicate Email	Owner token, already registered email	409 Conflict	Pass
TC-012	Non-Owner Role Attempt	Agent token, valid email	403 Forbidden	Pass
TC-013	Invalid Email Format	Owner token, malformed email	400 Bad Request	Pass

6.2.3 Savings Collection Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-018	Valid Savings Collection	Agent token, customerId, amount=500	Transaction created, balance updated, receipt returned; 201	Pass
TC-019	Zero Amount	Agent token, customerId, amount=0	400 Bad Request	Pass
TC-020	Negative Amount	Agent token, customerId, amount=-100	400 Bad Request	Pass
TC-021	Cross-Agent Customer	Agent A token, Agent B customer ID	403 Forbidden	Pass
TC-022	Balance Accuracy	Current balance=500, collection=100	New balance exactly 600	Pass

6.2.4 Loan and EMI Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-023	Valid Loan Application	Owner token, principal=10000, rate=12%, tenure=12	Loan doc PENDING; 201	Pass
TC-024	Loan Approval + EMI Schedule	Owner token, loanId	Loan ACTIVE; 12 installment docs created; 200	Pass
TC-025	EMI Calculation Accuracy	principal=12000, rate=12%, tenure=12	EMI = 1066.19	Pass
TC-026	Valid EMI Collection	Agent token, installmentId, amount=1066.19	Installment PAID; balance decremented; 201	Pass

TC-027	Loan Auto-Closure	Final EMI, outstandingBalance=0	Loan status updated to CLOSED	Pass
--------	-------------------	---------------------------------	-------------------------------	------

6.2.5 Tenant Isolation Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-029	Cross-Tenant Customer Read	Org A token, Org B customer ID	403 Forbidden	Pass
TC-030	Cross-Tenant Agent Read	Org A token, Org B agent ID	403 Forbidden	Pass
TC-031	Cross-Tenant Loan Read	Org A token, Org B loan ID	403 Forbidden	Pass
TC-032	Cross-Tenant Collection	Org A agent, Org B customer	403 Forbidden	Pass
TC-033	Suspended Org Request	Valid token, suspended organization	403 Forbidden	Pass
TC-034	No Org Context Token	Valid user token, no organization claim	403 Forbidden	Pass

6.2.6 Clerk Authentication Integration Test Cases

TC ID	Test Case	Input	Expected Output	Status
TC-035	Clerk User Provisioning — Agent	Owner calls agent creation API	Clerk user created via Admin API; credentials valid	Pass
TC-036	Clerk User Provisioning — Customer	Owner calls customer creation API	Clerk user created via Admin API; credentials valid	Pass
TC-037	Welcome Email Delivery	Agent/Customer creation event	Welcome email received with temporary password and login URL	Pass
TC-038	Password Change on First Login	User logs in with temporary password	Password change prompt displayed; new password accepted	Pass
TC-039	Org Membership Validation	Agent token without org membership	403 Forbidden	Pass

CHAPTER 7

RESULTS AND SNAPSHOTS

This chapter presents the FundCircle platform's key user interface screens and their functional descriptions, representing the completed implementation of every major module of the Pigmy Savings and Loan Collection Management System.

7.1 Landing Page



The FundCircle landing page presents the platform's value proposition to prospective Organization Owners. The hero section introduces FundCircle as a Pigmy Savings and Loan Collection Management System with a prominent 'Get Started' call-to-action leading to the Owner registration flow. Key feature highlights — Savings Management, Loan Management, and Agent Dashboard — are presented in a clean card layout. The page is fully responsive, adapting to mobile viewports for access from field locations.

7.2 Organization Owner Dashboard



The Owner Dashboard delivers real-time operational intelligence across eight KPI cards: Total Savings, Total Loans, Today's Collections, Active Customers, Active Agents, Pending EMIs, Overdue Loans, and Monthly Collection Summary (bar chart for the last twelve months). All KPIs update in real time via Firestore listeners.

7.3 Agent Dashboard



The Agent Dashboard is optimized for the daily collection workflow. It displays five operational panels: Assigned Customers, Today's Collections, EMI Collections, Pending Visits, and Recent Transactions. The mobile-responsive layout allows agents to use the dashboard efficiently on smartphones in the field.

7.4 Customer Dashboard



The Customer Dashboard provides complete financial transparency to account holders. It is organized into six sections: Savings Balance, Loan Balance, EMI Schedule, Collection History, Receipts, and Passbook. The dashboard requires no agent assistance — customers have direct self-service access to their complete financial history.

7.5 Customer Management Screen

The Customer Management screen is accessible only to Organization Owners. It presents a searchable data table of all customers within the organization with columns for Account Number, Customer Name, Assigned Agent, Savings Plan, Scheduled Amount, Current Savings Balance, Loan Status, and Account Status. A 'New Customer' button opens a creation form. On submission, the system provisions the Clerk user, creates the Firestore documents, and dispatches the welcome email.

7.6 Agent Management Screen

The Agent Management screen allows Organization Owners to view, create, and manage agent accounts. The 'New Agent' button opens a creation form requiring only the agent's email address. On submission, the backend provisions the Clerk user, creates the Firestore agent document, establishes the organization membership, and dispatches the welcome email with the temporary password.

7.7 Savings Collection Screen

The Savings Collection screen is the primary operational tool for agents. It features a customer search field with auto-complete, an amount input, and a current balance display. On submission, the system creates the Firestore transaction documents and immediately renders a print-ready digital receipt overlay containing: receipt number, organization name, customer name, account number, collected amount, new savings balance, collection timestamp, and recording agent name.

7.8 Loan Management Screen

The Loan Management screen is accessible to Organization Owners and covers the complete loan lifecycle. The screen lists all loans with their status, customer name, principal amount, outstanding

balance, and next due date. Owners can approve or reject pending loan applications and view auto-generated EMI schedules for active loans.

7.9 EMI Collection Screen



The EMI Collection screen enables agents to record loan repayment collections in the field. The agent searches for a customer with an active loan, and the screen displays the loan summary with the next due installment. On submission, the system atomically updates the Firestore documents, generates a receipt, and automatically closes the loan if the outstanding balance reaches zero.

7.10 Reports Screen



The Reports screen provides Organization Owners with analytical visibility into business performance. It offers four report categories: Monthly Collection Summary, Loan Portfolio Report, EMI Aging Report, and Customer Savings Growth Report. All reports support date-range filtering and are rendered with interactive charts.

— End of Report —